SHIT Sciences Humanities Information Technology

构石

基于 Git 的论文发布与评审规则设计

WMWYT[1]，豆包[2]
其他大学
通讯作者：WMWYT（e-mail: 1060274259@qq.com）
本研究由本人基金资助（项目编号：9527）
本文包含补充材料

ABSTRACT / 摘要
本研究针对 S.H.I.T 平台去中心化论文评审缺少实现平台、期刊落地难度大的问题，结合软件开发中 Git 工具的版本管理逻辑，设计了一套适配 Git 服务器的去中心化论文发布与评审规则。该规则将 S.H.I.T 作为预印本发布载体，以 GitHub 仓库为核心评审阵地，按“预印本发布 - 深度发酵 - 正式出版”的流程，把仓库划分为展示论文的“第一池”、提出评审异议的 Issue“第二池”、定稿发布的 Releases“第三池”，并制定了百分制量化的“无害化”评审标准、审稿人交互规则及版本修改次数限制等具体评审要求。研究还分析了该规则在价值度量、见刊判断、学术不端降解和资源配置四大模块的落地效果，同时指出规则在理性参与假设、算法适配性、匿名评审、成果归属及去中心化边界等方面存在的问题，为 S.H.I.T 平台的去中心化学术评审提供了一套可落地的简易实施方案。

INDEX TERMS / 关键词 去中心化平台，版本管理系统，Git。

IMPACT STATEMENT / 影响声明 本研究依托 Git 版本管理工具的核心功能，为 S.H.I.T 平台设计了一套适配性的去中心化论文发布与评审实操规则，尝试解决去中心化学术评审落地过程中平台缺失、流程不规范的现实问题，为学术评审模式的创新探索提供了一套可参考的简易实施方案。
ADDRESS / 论文地址 https://github.com/WMWYT/shit_journal_review_rules.git

I. 引言

自 S.H.I.T 提出社区治理板块以来，已出现了几篇高质量的去中心化学术评审方案，一些方案完全具备可行性，但缺少实现平台，加上期刊人手问题，若从头实现一套去中心化架构的论文评审系统难度过大。

基于此背景，本文提出一套规则，通过使用现有大公司的 git 服务器发表论文。通过 Issues 提出异议和判断论文是否通过评审，以最后 Releases 版本作为论文最终版定稿的流程规则，实现去中心化的发布和评审方案。

根据作者末流大学的本科软件工程学术经历和一年半的工程师助理工作经验，设计了一套适配 Git 的论文发布和评审的规则。

现如今论文的发布流程一般为编写论文，选择投稿，期刊编辑初审后进行同行评审，如果出现问题再返修，论文通过后，期刊录用出版。而如果为了抢首发，作者会发布到预印本中，之后在正式期刊发布。

而在 S.H.I.T 中，观察到的模式是希望期刊做预印本又通过大众和同行评审发布正式期刊。那么我们可以将论文的发布流程归纳为，作者直接选定期刊发布预印本，预印本被大众和同行看到之后，提出修改意见，修改至可发布后，发布正式版本出版。

在确认了流程之后，发现与软件代码开发中需要使用到 Git 时的流程十分相似。在代码开发过程中，也是开发人员发布代码，经过组内成员讨论，反复修改，最后上线。所以作者认为可以通过借鉴软件代码开发版本管理工具 Git 来处理论文的发布和评审流程。

期刊平台可以只提供去中心化的评审规则，通过适配Git功能，对每位用户的论文发布、修改和评审环节只需要在仓库中解决。

II. 正文

基于现阶段平台只有发表在S.H.I.T论文的信息收集、展示和评分功能。加上作者的一套评价规则（现阶段本论文作者所有在S.H.I.T发表的社区治理规则论文都没看懂，所以自创一个，不一定能在后续适配上社区中提供的去中心化算法），搭配GitHub，实现一套简易的去中心化论文评审方案。

预印本发布：

论文需要先在S.H.I.T上发布一版作为预印本，标题为”论文题目-化粪池”，同时论文作者将在GitHub上新建一个存储论文的仓库作为化粪池的实体，在的影响声明下附论文地址。将论文的原件和”.pdf”版本Push到仓库中。当预印本进入化粪池后，代表论文可以继续发酵，嗅探兽可以从论文中看到仓库地址。在其中必须添加一个”readme.md”，此文件只用于展示论文版本，格式如下：

```
# V 1
上传第一版。
下版本修改：
xx修改为xx
# V 2
修改了xx
修改了xx
下版本修改：
xx修改为xx
# V 3
……
```

深度发酵：

依据现实中的化粪池结构，将仓库分为三个部分，仓库首页作为第一池，用于展示论文，用于给嗅探兽拿到自己想要格式的论文，顺便也解决了现阶段的论文下载功能。Issue作为第二池，用于论文的深度发酵，嗅探兽提出Issue作为消灭论文中的问题。Releases作为第三池，用于进一步沉淀确保论文以达到无害化标准。

出版：

在达到无害化标准之后，作者将重新在S.H.I.T上传一份论文，作为论文的正式版本，这回标题将只有论文题目。管理员撤回或作者自行删除化粪池版。

通过此规则流程基本解决了资源配置模块中技术运行方案的问题，只要GitHub还在，就算平台崩溃了，还可以依照此方法找回论文，甚至为了保险起见，还可以在多个商业Git仓库中拉取项目论文，做到分布式备份数据。

而提交论文后如何实现去中心化的评价体系和方法，才是其核心讨论观点。为实现此目标，需要了解到现有体系下，我们能得到的可以用来评分的参数，从而来判断一篇论文是否达到了无公害的标准。

现阶段论文是否达到无公害标准，采用百分分制，其计算方法十分简单。无公害标准为当前论文版本下的”documentation”标签中”yes”总数占60%以上（含60%，但为什么为60%没有理由，只是在大部分考试时都是认为60分及格）。

以现阶段S.H.I.T官网提供的功能来规定，只有一个满分为5分的评分系统，由于网站的特殊性，其评分所能带来的价值比较难以定义，在现有环境下先用着。好在当上传到Git仓库之后，作者可以对文章进行修改，审稿人可以提交Issue，这些操作可以为评分系统的构建带来一些可以量化的评价。

在这里规定，当提交到GitHub上之后，论文作者马上在第二池中添加Issue，建立”documentation”标签的主题，此标签在池中只能由论文作者建立，每次提交新版本的论文前只能存在一个，用于审稿人同意达到无害化标准，此标签下审稿人，只能回复”yes”和”no”，”yes”代表达到无害化，”no”代表，此标签用于表示论文是否能达到第三池用于出版。

若审稿人希望在此标签下回复，审稿人需要先新建一个Issue，如果不认可使用”bug”标签，并在Issues中按照常见的提交Bug问题方式指出其问题，如果认可发送一个无标签标题为”LGTM”（Looks Good To Me），内容中只写”yes”的Issue。

完成此项工作后，审稿人再在”documentation”中回复，提交”LGTM”回复”yes”，提交”bug”标签内容回复”no”，所有在规定标签下回复的固定内容大小写敏感。做完此工作后审稿人将不在当前论文版本的第二池中新建任何Issue，第二池中只能有这三种格式的Issue，其他任何格式，作者都可以无视，方便管理和统计。

当标签出现后，论文作者需要到仓库中确认，确认方式为关闭Issue。如果为”LGTM”,论文作者可直接关闭。如果为”bug”标签，论文作者可在Issue中与审稿人讨论解决办法，是修改论文，还是作者解释其这么做的原因。当此Issue确认好解决办法之后，审稿人在最后一次回复”yes”，每个”bug”中只允许审稿人对于作者的回答提交3次继续的提问，且问题交流只限于发送标题的审稿人和作者。在五次评论无效后，审稿人回复”no”，此回复不在5次交流中，表明审稿人对此论文评价为不满意。此时作者关闭此Issue。但在”bug”中如果识别次问题是否为讨论论文问题，而不是来捣乱的，此时再引入第三方评价，此时为对此问题的评价其他评审人和论文作者对此问题发送表情，大拇指向上表情为认可，大拇指向下为不认可。再此评论的2小时内，两种表情的数量大于3，且向上的表情数大于向下的表情为认可此问题需要论文作者解决或解释，作者必须做出回答。其他表情为无效表情。如果两小时后两种表情数小于3，或两小时后向下表情大于向上表情，论文作者可不回答此问题，回复一个”no”用于标记时间，之后可直接关闭此Issue。

当”documentation”标签中的评论数量大于 20 个时，统计前 20 个评论，这些评论必须关闭对应的 Issue。或当前版本的 5 个工作日内以关闭所有 Issue，且数量小于 20 没有新的模板 Issue 提出。

统计其中”yes”数量，当大于等于 12 时，说明此篇论文以达到无害化标准，并关闭此”documentation”标签的讨论，论文作者此时可以决定将这篇论文发布为 Releases 作为出版版本，也可以依照评论区中的意见再次修改上传新版本论文，并在 Readme 文件中说明修改的内容。如果小于 12，论文作者必须按照 Issue 区中的讨论给出结果，上传修改过后的论文，并在 Readme 中编写好修改信息。在第二池中建立新的”documentation”标签 Issue 用于讨论新版本论文中是否达到了无害化标准。每篇论文只有 3 次版本修改机会，3 次都没通过，说明论文争议过大。3 次修改中有一次通过就可以依照发布出版流程公布正式版本。

管理员可依据这整个论文的评审流程决定此论文在网站上的展示方式。任何关闭 Issue 的论文都视为不可信任。

III. 讨论

本论文在现有的功能框架下，搭建了一套为达到去中心化论文评审目标而搭建的论文评审规则。在编写这些规则时，都是基于所有参与的人是在理性的情况下做出判断，这几乎是不可能。但软件开发都是先干了在说，后面的事，后面再说。

现有的规则框架下，由于作者水平不高，那些高深的去中心化算法看不懂，不知道现在制定的规则能否适配那些去中心化算法，在规则制定的过程中只考虑了功能是否自洽，未考虑这些规则的扩展性。考虑到后续需要将这些规则变为代码实现自动化处理，这也是个大麻烦。

基于此规则和 GitHub 实现的方法，还无法做到匿名评审。但论文这一文本形式，或许一开始就不应该匿名评审。

在软件开发领域，将代码上传至 Git 仓库并开放出来，基本等同于将自己的研究成果开源出来，但代码的归属还是能由开发者掌握，但论文放在 Git 仓库中开放评审，其研究成果是多方面的涵盖了很多其他方面的学科。若以代码的开源协议来规定论文的成果归属，也无法保证这些协议能起到保护作者功劳的结果。现阶段以此种方式做论文评审的作者，请考虑好后续论文发表后产生的问题。

论文编写时，考虑到一个问题，去中心化的边界在哪。在实际生活中发现，如果真的去中心化之后每个人就需要具备更多技能去存储和表达自己的数据，但很多人没有如此多的时间去学习新技能，最后又去拥抱中心化的方法。以现在最流行的去中心化产品，各个虚拟货币为例，真正有能力挖矿的又有多少人，大部分人还是在几个集中的交易市场上买卖。所以 S.H.I.T 的去中心化要如果实现，在公平和效率上要如何取舍。

IV. 结论

本文提出了一种用于解决现阶段 S.H.I.T 的去中心化评审方案。基本解决了学术去中心化方案的 4 个模块[1]问题。

价值度量模块：

借助 Git 工具上传到仓库，可对每一次操作进行详细的记录，流程透明，只要有一个”小孩”指出”皇帝”没穿衣服，只要不是所有人在表演，就会有人去验证论文的可靠性。

见刊判断模块：

通过审稿人对本论文提出的意见反馈，通过最直接的，大众审稿人是否认为这篇文章可以通过评审，做到其自动降解与晋升。

学术不端自动降解系统：

由于学术论文的特殊性，出现学术不端问题的论文，通常会出现大量的可识别的证据，只要发现除非审稿人有利益相关，将其报出来的收益可能会更大。由于整个流程可追踪，在 Bug 中审稿人可提出其学术不端的问题。

资源配置模块：

现阶段的所有操作都还是停留在手工统计的方法，基本是做到了最低。但现阶段没有设置社区激励方式，由于 S.H.I.T 的特点，能看这些论文甚至还能评价已经算是一种激励。以现在的流量来看，微软公司，还是能应付过来的。

致谢

感谢微软愿意在不知情的情况下为本使用提供 git 仓库支持。感谢豆包对本研究数据的挖掘和整理工作，感谢自己的离职申请，放下了工作好轻松。

REFERENCES

[1] Dr.S.H.I.T，魔芋 OwO，S.H.I.T 治理公约 1.0：学术去中心化方案征稿，https://shitjournal.org/news/governance-1.0